RTOS

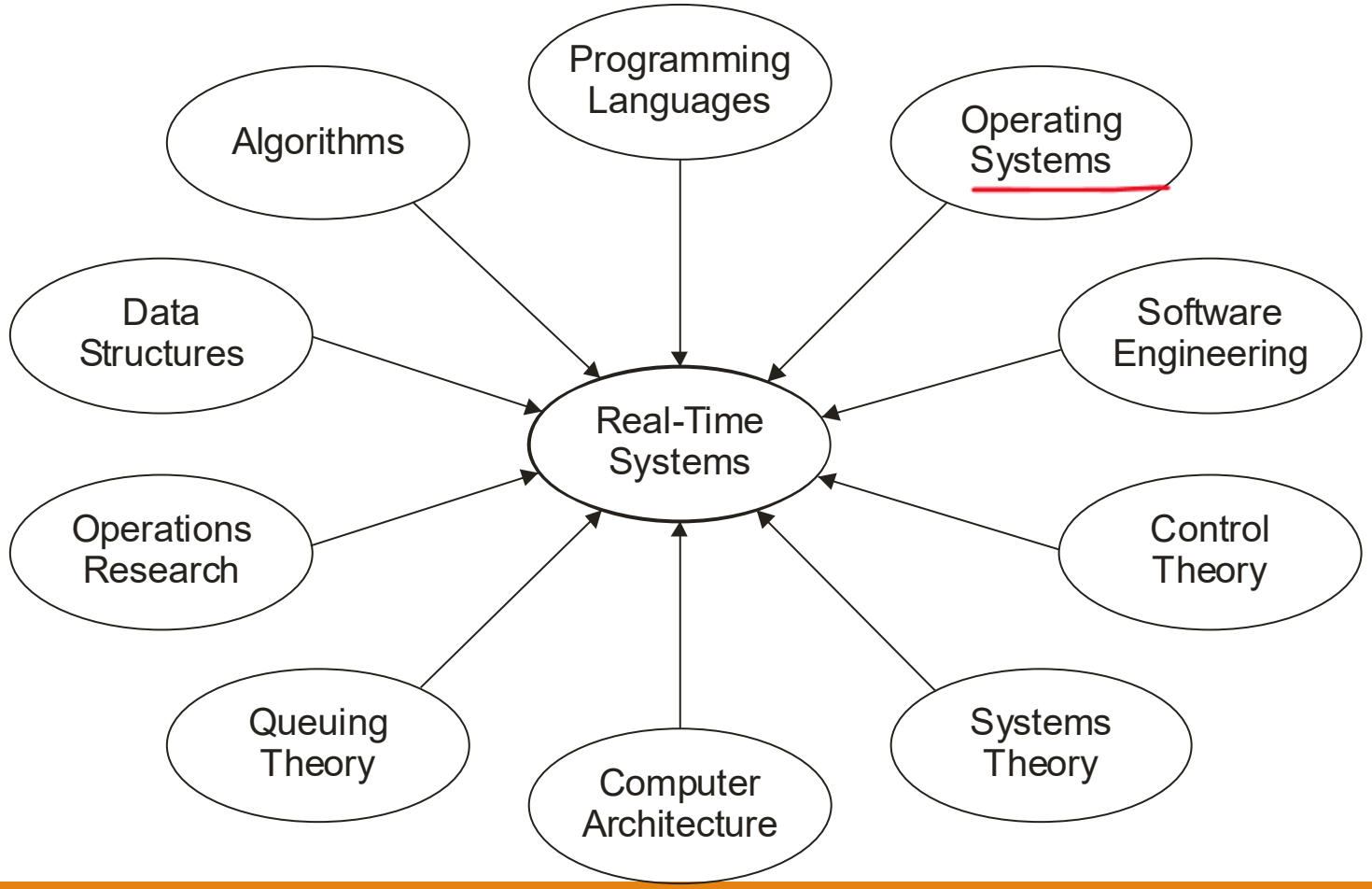
REFERENCES:
HTTPS://STAFF.EMU.EDU.TR/ALEXANDERCHEFRANOV/DOCUMENTS/CMSE443/CMSE443%20SPRING2020/LAPLANTE2012%20REAL-TIME%20SYSTEMS%20DESIGN%20AND%20ANALYSIS.PDF

HTTPS://NPTEL.AC.IN/COURSES/106105172

Outline

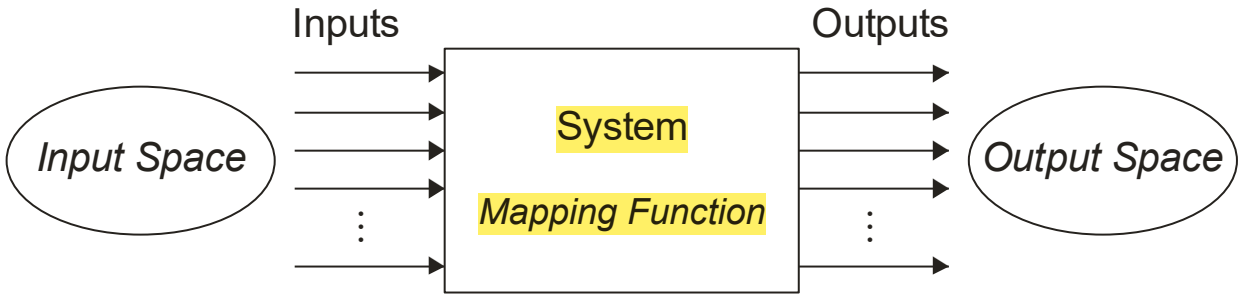
- Fundamentals of Real-Time Systems
- RTOS Classification
- RTOS Scheduling

Rich Variety of “Real-Time” Disciplines



Definition: System

A system is a mapping of a set of inputs into a set of outputs



- A system is an assembly of components connected together in an organized way
- A system is fundamentally altered if a component joins or leaves it
- It has a purpose
- It has a degree of permanence
- It has been defined as being of particular interest

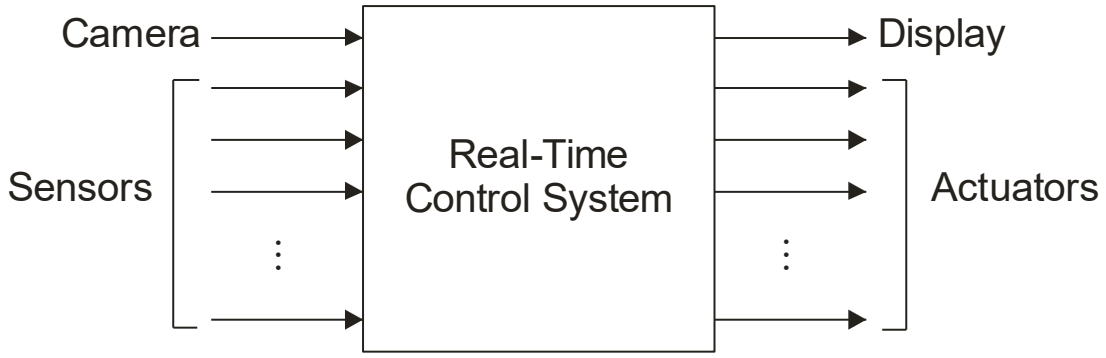
Example: A Real-Time Control System

Inputs are *excitations* and outputs are corresponding *responses*

Inputs and outputs may be digital or analog

Inputs are associated with sensors, cameras, etc.

Outputs with actuators, displays, etc.



Either response time statisfies bounded response time constraints or it will risk consequences

Definitions: Real-Time System

A real-time system is a computer system that must satisfy bounded response-time constraints or risk severe consequences, including failure

A real-time system is one whose logical correctness is based on both the correctness of the outputs and their timeliness

Definition: Response Time

The time between the presentation of a set of inputs to a system and the realization of the required behavior, including the availability of all associated outputs, is called the response time of the system

How fast and punctual does it need to be?

- Depends on the specific real-time system

Definition: Failed System

A failed system is a system that cannot satisfy one or more of the requirements stipulated in the system requirements specification

Hence, rigorous specification of the system operating criteria, including timing constraints, is necessary

Soft, Hard, and Firm “Real-Time”

Definition: Soft Real-Time System

A soft real-time system is one in which performance is degraded but not destroyed by failure to meet response-time constraints

Definition: Hard Real-Time System

A hard real-time system is one in which failure to meet even a single deadline may lead to complete or catastrophic system failure

Definition: Firm Real-Time System

A firm real-time system is one in which a few missed deadlines will not lead to total failure, but missing more than a few may lead to complete or catastrophic system failure

Example: Real-Time Classification

System	Real-Time Classification	Explanation
Avionics weapons delivery system in which pressing a button launches an air-to-air missile	Hard	Missing the deadline to launch the missile within a specified time after pressing the button may cause the target to be missed, which will result in a catastrophe
Navigation controller for an autonomous weed-killer robot	Firm	Missing a few navigation deadlines causes the robot to veer out from a planned path and damage some crops
Console hockey game	Soft	Missing even several deadlines will only degrade performance

Definitions: Event and Release Time

Definition: Event : Occurrence that alters the sequential execution of program counter

Any occurrence that causes the program counter to change non-sequentially is considered a change of flow-of-control, and thus an event

Definition: Release Time

The release time is the time at which an instance of a scheduled task is ready to run, and is generally associated with an interrupt

Taxonomy of Events

An event can be either synchronous or asynchronous

- ***Synchronous*** events are those that occur at predictable times in the flow-of-control
- ***Asynchronous*** events occur at unpredictable points in the flow-of-control and are usually caused by external sources

Moreover, events can be **periodic**, **aperiodic** or **sporadic**

- A real-time clock that pulses regularly is a *periodic* event
- Events that do not occur at regular periods are called *aperiodic*
- Aperiodic events that tend to occur very infrequently are called *sporadic*

Example: Various Types of Events

Type	Periodic	Aperiodic	Sporadic
Synchronous	Cyclic code	Conditional branch	Divide-by-zero (trap) interrupt
Asynchronous (due to external source)	Clock interrupt	Regular, but not fixed-period interrupt	Power-loss alarm

Deterministic Systems

For any physical system, certain states exist under which the system is considered to be out of control

The software controlling such a system must therefore avoid these states

In embedded real-time systems, maintaining overall control is extremely important

Software control of any real-time system and associated hardware is maintained when the next state of the system, given the current state and a set of inputs, is predictable

Definition: Deterministic System

A system is deterministic, if for each possible state and each set of inputs, a unique set of outputs and next state of the system can be determined

CPU Utilization or Time-Loading Factor

The final term to be defined is a critical measure of real-time system performance

Because the CPU continues to execute instructions as long as power is applied, it will more or less frequently execute instructions that are not related to the fulfillment of a specific deadline

The measure of the relative time spent doing *non-idle processing* indicates how much real-time processing is occurring

Definition: CPU Utilization Factor

The CPU utilization or time-loading factor, *U*, is a relative measure of the non-idle processing taking place

Calculation of *U*

U is calculated by summing the contribution of utilization factors for each task

Suppose a system has $n \geq 1$ periodic tasks, each with an execution period of p_i

If task *i* is known to have a *worst-case* execution time of e_i, then the utilization factor, u_i, for task *i* is

$$u_i = e_i/p_i \qquad (1.1)$$

Furthermore, the overall system utilization factor is

$$U = \sum_{i=1}^{n} u_i = \sum_{i=1}^{n} e_i/p_i \quad (1.2)$$

In practice, the determination of e_i can be difficult, in which case estimation or measuring must be used

For aperiodic and sporadic tasks, u_i is calculated by assuming a worst-case execution period

Example: Calculation of *U*

Suppose, an individual elevator controller in a bank of elevators has the following tasks with execution periods of p_i and worst-case execution times of e_i, $i \in [1,2,3,4]$:

Task 1: Communicate with the group dispatcher.

Task 2: Update the car position information and manage floor-to-floor runs as well as door control.

Task 3: Register and cancel car calls.

Task 4: Miscellaneous system supervisions.

i	e_i	p_i
1	17 ms	500 ms
2	4 ms	25 ms
3	1 ms	75 ms
4	20 ms	200 ms

$U = \sum_{i=1}^{4} e_i/p_i$=0.31

31% (Very safe zone)

Diversifying Applications

The history of real-time systems is tied inherently to the evolution of computer

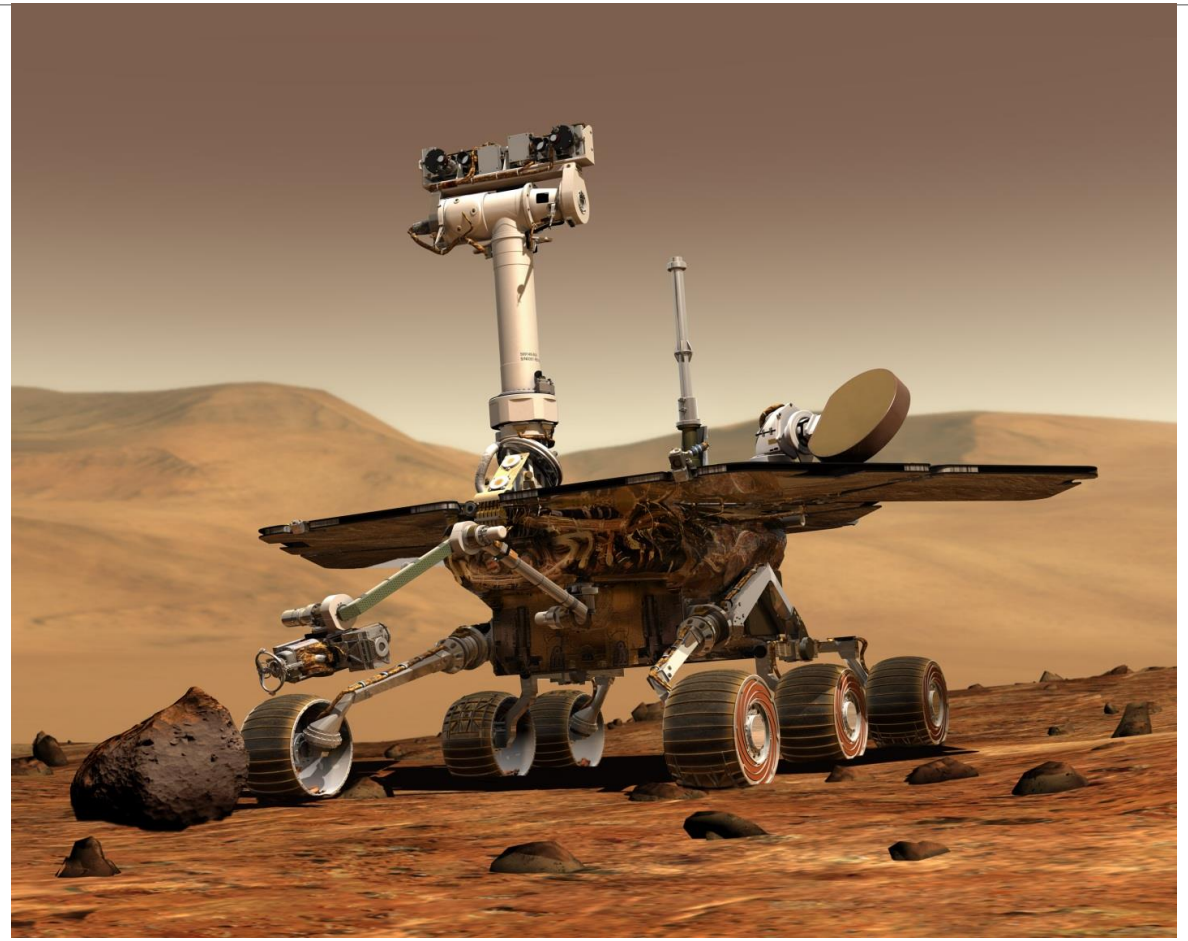
Modern real-time systems, such as those that control nuclear power plants, are highly sophisticated

- Nevertheless, they may still exhibit characteristics of those pioneering systems developed in the 1940s – 1960s

An excellent example of an advanced real-time system is the Mars Exploration Rover of NASA

- An autonomous system with extreme reliability requirements
- Receives commands and sends measurement data over radio-communications links
- Performs its scientific missions with the aid of multiple sensors, processors, and actuators

Mars Exploration Rover of NASA



Practical Embedded Systems

Aerospace
- Flight control
- Navigation
- Pilot interface

Automotive
- Airbag deployment
- Antilock braking
- Fuel injection

Household
- Microwave oven
- Rice cooker
- Washing machine

Industrial
- Crane
- Paper machine
- Welding robot

Multimedia
- Console game
- Home theater
- Simulator

Medical
- Intensive care monitor
- Magnetic resonance imaging
- Remote surgery

RTOS Scheduling

3.2 Classification of Real-Time Task Scheduling Algorithms

Several schemes of classification of real-time task scheduling algorithms exist. A popular scheme classifies the real-time task scheduling algorithms based on how the scheduling points are defined. The three main types of schedulers according to this classification scheme are: clock-driven, event-driven, and hybrid.

> The clock-driven schedulers are those in which the scheduling points are determined by the interrupts received from a clock. In the event-driven ones, the scheduling points are defined by certain events which precludes clock interrupts. The hybrid ones use both clock interrupts as well as event occurrences to define their scheduling points.

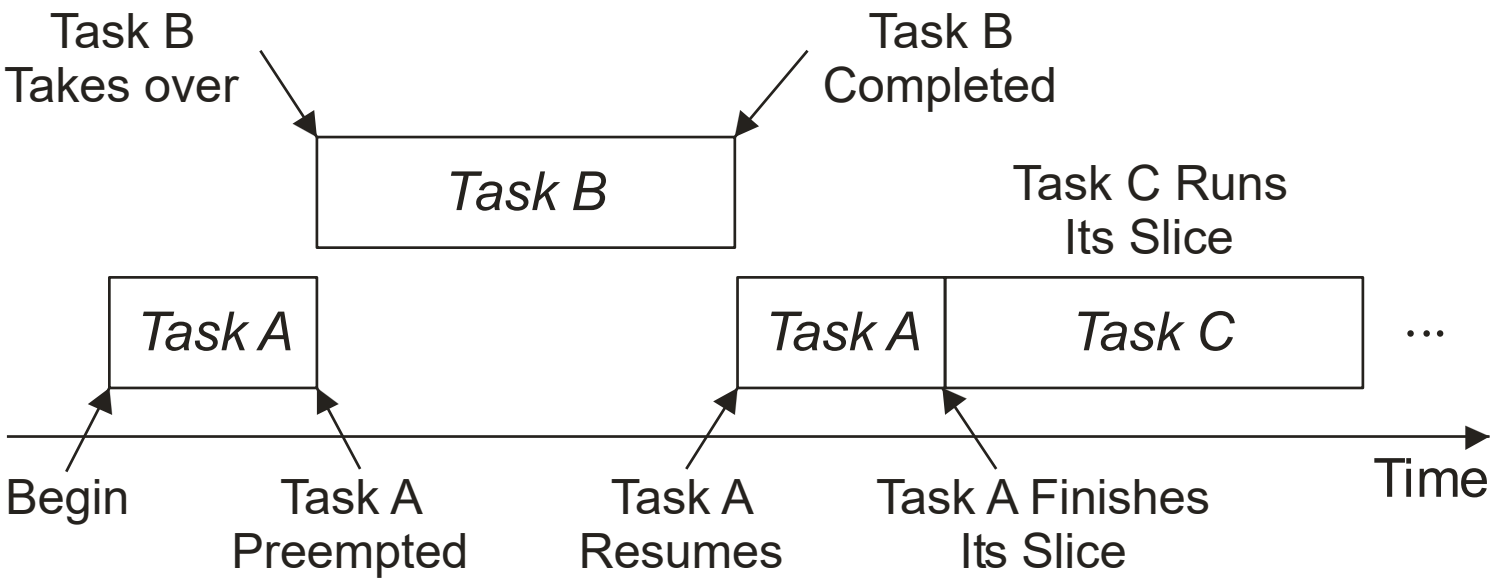
A few important members of each of these three broad classes of scheduling algorithms are the following:

1. Clock Driven:
   - Table-driven
   - Cyclic
2. Event Driven:
   - Simple priority-based
   - Rate Monotonic Analysis (RMA)
   - Earliest Deadline First (EDF)
3. Hybrid:
   - Round-robin

Round-robin scheduling

- In a round-robin system several processes are executed sequentially to completion, often in conjunction with a cyclic executive.
- In round-robin systems with time slicing, each executable task is assigned a fixed-time quantum called a time slice in which to execute.
- A fixed-rate clock is used to initiate an interrupt at a rate corresponding to the time slice.
- The task executes until it completes or its execution time expires, as indicated by the clock interrupt.
- If the task does not execute to completion, its context must be saved. The task is the placed at the end of the executable list. The context of the next executable task in the list is restored, and it resumes execution.
- Round-robin systems can be combined with pre-emptive priority systems, yielding a kind of mixed system.

Round-robin scheduling…



Mixed scheduling of three tasks. Here process A and C are of the same priority, whereas B is of higher priority. Process A is executing for some time when it is preempted by task B, which executes until completion. When process A resumes, it continues until its time slice expires, at which time context is switched to process C, which begins executing.

For PERIODIC tasks :

Cyclic executives

- A simple approach that generates a complete and highly predictable schedule.
- The CE refers to a scheduler that deterministically interleaves and sequentializes the execution of periodic tasks on a processor according to a pre-run-time schedule.
- In general terms, the CE is a table of procedure calls, where each task is a procedure, and it executes a single do loop.
- Scheduling decisions are made periodically, rather than at arbitrary times.
- Time intervals during scheduling decision points are referred to as frames or minor cycles, and every frame has a length f called the frame size.

Cyclic executives…

The major cycle is the minimum time required to execute tasks allocated to the processor ensuring the deadlines and periods of all processes are met.

The major cycle or the hyperperiod is equal to the least common multiple of the periods, i.e., lcm(p_1, … p_n).

As scheduling decisions are made only at the beginning of every frame, there is no preemption within each frame.

The phase of each periodic task is a nonnegative integer multiple of the frame size.

The scheduler carries out monitoring and enforcement actions at the beginning of each frame.

Cyclic executives...

Frames must be sufficiently long so that every task can start and complete within a single frame. This implies that the frame size, f, is to be longer than the execution time, e_i, of every task, τ_i, that is,

$$C_1 : f \geq \max_{i \in [1,n]} (e_i). \tag{3.4}$$

In order to keep the length of the cyclic schedule as short as possible, the frame size, f, should be chosen so that the hyperperiod has an integer number of frames:

$$C_2 : \lfloor p_{\text{hyper}} / f \rfloor - p_{\text{hyper}} / f = 0. \tag{3.5}$$

Moreover, to ensure that every task completes by its deadline, frames must be short so that between the release time and deadline of every task, there is at least one frame. The following relation is derived for a worst-case scenario, which occurs when the period of a task starts just after the beginning of a frame, and, consequently, the task cannot be released until the next frame:

$$C_3 : 2f - \gcd(p_i, f) \leq D_i. \tag{3.6}$$

where "gcd" is the greatest common divisor, and D_i is the relative deadline of task τ_i. This condition should be evaluated for all schedulable tasks.

Cyclic executives

To illustrate the calculation of the framesize, consider the set of tasks

τ_i	p_i	e_i	D_i
τ_2	15	1	14
τ_3	20	2	26
τ_4	22	3	22

The hyperperiod is equal to 660 since the least common multiple of 15, 20, and 22 is 660. The three conditions mentioned above are evaluated as follows:

$$C_1 : \forall i \; f \geq e_i \;\Rightarrow\; f \geq 3$$

$$C_2 : \lfloor p_i / f \rfloor - p_i / f = 0 \;\Rightarrow\; f = 2,3,4,5,10,\ldots$$

$$C_3 : 2f - \gcd(p_i, f) \leq D_i \;\Rightarrow\; f = 2,3,4,5$$

From these three conditions, it can be inferred that a possible value for f could be any one of the values of 3, 4, or 5.

Rate-monotonic scheduling

Theorem (Rate-monotonic) [Liu and Layland '73]

Given a set of periodic tasks and preemptive priority scheduling, then by assigning priorities such that the tasks with shorter periods have higher priorities (rate monotonic) yields an optimal scheduling algorithm. RMA bound (may not **always** hold)

Theorem (RMA Bound)

Any set of n periodic tasks is RM-schedulable if the processor utilization, U, is no greater than $n\left(2^{\frac{1}{n}}-1\right)$.

This means that whenever U is at or below the given utilization bound, a schedule can be constructed with RM. In the limit when the number of tasks $n=\infty$, the maximum utilization limit is

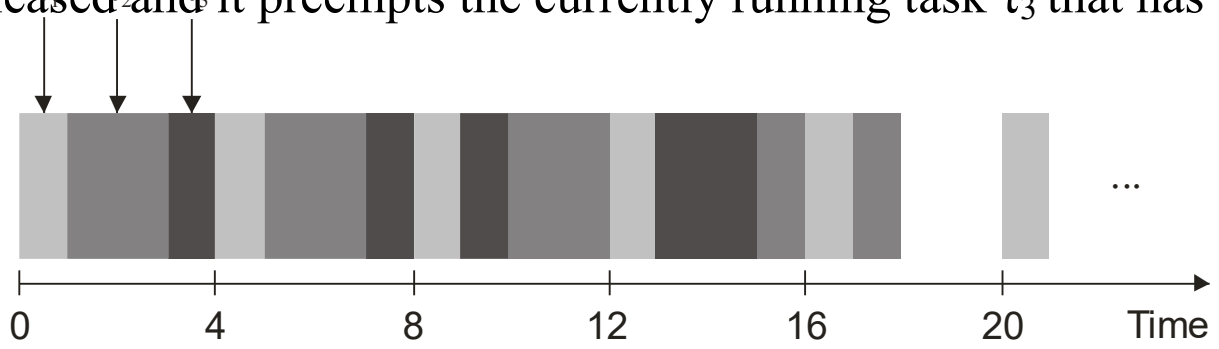
$$\lim_{n\to\infty} n\left(2^{\frac{1}{n}}-1\right)=\ln 2 \approx 0.69$$

Rate-monotonic scheduling

To illustrate rate-monotonic scheduling, consider the task set shown

τ_i	e_i	p_i	$u_i = e_i/p_i$
τ_1	1	4	0.25
τ_2	2	5	0.4
τ_3	5	20	0.25

. All tasks are released at time 0. Since task τ_1 has the smallest period, it is the highest priority task and is scheduled first. Note that at time 4 the second instance of task τ_1 is released and it preempts the currently running task τ_3 that has the lowest priority.



Rate-monotonic scheduling…

Not every system is appropriate as rate-monotonic.

Consider nuclear plant control system – should visual display have highest priority?

What about aperiodic and sporadic tasks?

In these cases RM used a feasibility check.

Earliest deadline first

In contrast to fixed-priority algorithms, in dynamic priority schemes the priority of the task with respect to that of the other tasks changes as tasks are released and completed.

One of the most well-known dynamic algorithm, earliest-deadline-first (EDF), deals with deadlines rather than execution times.

The ready task with the earliest deadline has the highest priority at any point of time.

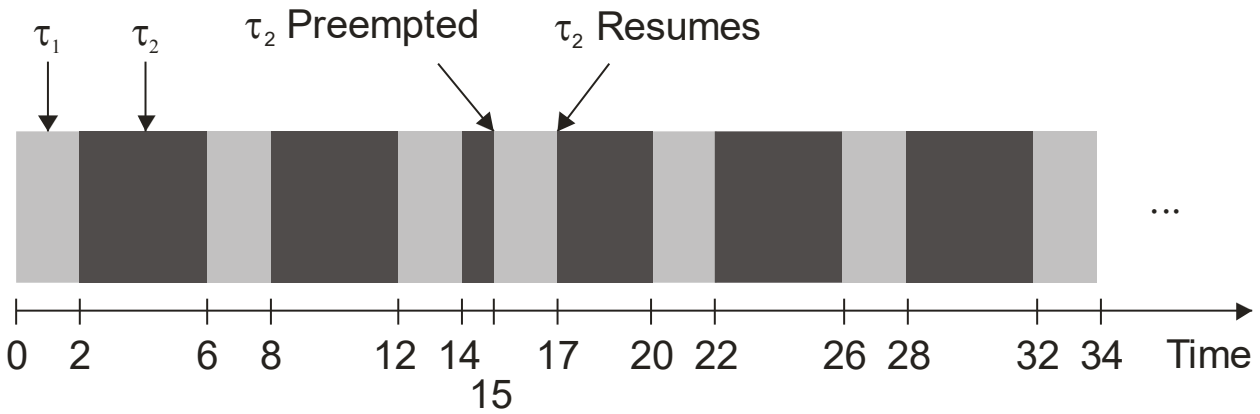
Earliest deadline first

Theorem [EDF Bound]

A set of n periodic tasks, each of whose relative deadline equals its period, can be feasibly scheduled by EDF if and only if

$$\sum_{i=1}^{n} \left(e_i / p_i \right) \leq 1$$

EDF example



Task set

τ_i	e_i	p_i
τ_1	2	5
τ_2	4	7

EDF schedule for task set shown. Although τ_1 and τ_2 release simultaneously, τ_1 executes first because its deadline is earliest. At $t = 2$, τ_2 can execute. Even though τ_1 releases again at $t = 5$, its deadline is not earlier than τ_3's. This sequence continues until time $t = 15$ when τ_2 is preempted as its deadline is later ($t = 21$) than τ_1's ($t = 20$). τ_2 resumes when τ_1 completes.